

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Майстренко Анастасия Николаевна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Выполнить контейнеризацию микросервисного приложения сервиса аренды недвижимости средствами Docker:

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Ход работы

1. Dockerfile для каждого сервиса

Для каждого из пяти компонентов (Identity, Catalog, Booking, Messaging и API Gateway) создан отдельный Dockerfile. Контекст сборки — корень проекта, поэтому в образ копируются общий код (shared/) и код конкретного сервиса. Зависимости устанавливаются отдельным слоем — он кэшируется и переиспользуется при сборке остальных сервисов.

```
FROM node:20
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY tsconfig.json ./
COPY shared ./shared
COPY services/catalog ./services/catalog
EXPOSE 8002
CMD ["npm", "run", "start", "services/catalog/app.ts"]
```

Файл .dockerignore исключает node_modules, dist, db и .git, чтобы образы были компактнее и сборка быстрее.

2. Общий docker-compose.yml

Общий файл docker-compose.yml собирает образы всех сервисов и поднимает их вместе с инфраструктурой — PostgreSQL и RabbitMQ. Состав приведён в таблице 1.

Таблица 1 — Сервисы в docker-compose

Сервис	Образ/сборка	Назначение
db	postgres:16	БД-на-сервис (init-dbs.sql создаёт 4 базы)
rabbitmq	rabbitmq:3-management	Брокер сообщений (AMQP + панель :15672)
identity	services/identity/Dockerfile	Аутентификация и пользователи
catalog	services/catalog/Dockerfile	Объекты и удобства
booking	services/booking/Dockerfile	Сделки и отзывы
messaging	services/messaging/Dockerfile	Диалоги и сообщения

gateway	gateway/Dockerfile	Единая точка входа (порт 8000)
---------	--------------------	--------------------------------

Для PostgreSQL и RabbitMQ настроены healthcheck-и, а сервисы через depends_on с condition: service_healthy запускаются только после готовности инфраструктуры. Весь запуск выполняется одной командой:

```
docker compose up -d --build
```

3. Сетевое взаимодействие

Все контейнеры подключены к общей bridge-сети rental-net. Внутри неё Docker предоставляет DNS по имени сервиса, поэтому компоненты обращаются друг к другу не по localhost, а по именам контейнеров. Соответствие адресов задано через переменные окружения в docker-compose (таблица 2).

Таблица 2 — Сетевые адреса внутри Docker-сети

Откуда	Адрес назначения	Назначение
gateway	http://identity:8001 и др.	Проксирование на сервисы
booking	http://catalog:8002	Синхронный /internal-запрос
все сервисы	amqp://rabbitmq:5672	Очереди сообщений
все сервисы	PG_HOST=db	Подключение к PostgreSQL

Наружу (на хост) открыт только порт API Gateway — 8000; остальные сервисы доступны лишь внутри сети, что повышает безопасность и соответствует роли шлюза как единой точки входа.

4. Проверка работоспособности

Образы всех пяти сервисов успешно собраны (docker compose build), корректность docker-compose.yml подтверждена командой docker compose config. Стек запущен в контейнерах, и через API Gateway выполнен сквозной сценарий — все взаимодействия прошли по именам сервисов в Docker-сети (таблица 3).

Таблица 3 — Результаты проверки в контейнерах

Проверка	Результат
Сборка 5 образов (docker compose build)	успешно
Валидация docker-compose.yml (docker compose config)	valid
Запуск стека, gateway отвечает на /health	ok
gateway → identity по имени (регистрация/вход)	ok
gateway → catalog по имени (объект, поиск)	ok
booking → http://catalog:8002 (цена при бронировании)	7500, ok
событие booking.confirmed через amqp://rabbitmq	доставлено

Catalog по событию: объект → rented	rented
Messaging по событию: открыт диалог	создан

Таким образом, всё приложение запускается в контейнерах одной командой, а сервисы корректно взаимодействуют между собой по сети Docker — синхронно (REST) и асинхронно (RabbitMQ).

Вывод

В ходе лабораторной работы выполнена контейнеризация микросервисного приложения сервиса аренды недвижимости. Для каждого сервиса реализован собственный Dockerfile, написан общий docker-compose.yml, объединяющий сервисы с PostgreSQL и RabbitMQ, и настроено сетевое взаимодействие через общую bridge-сеть с обращением по именам сервисов.

Всё приложение разворачивается одной командой `docker compose up`. Наружу открыта единственная точка входа — API Gateway, остальные сервисы изолированы во внутренней сети. Работоспособность подтверждена сквозным сценарием, в котором сервисы успешно взаимодействуют по сети Docker как синхронно, так и через очереди сообщений.